

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 2: Entity-Relationship Modelling

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO1
Session	Skilling 2	Date	

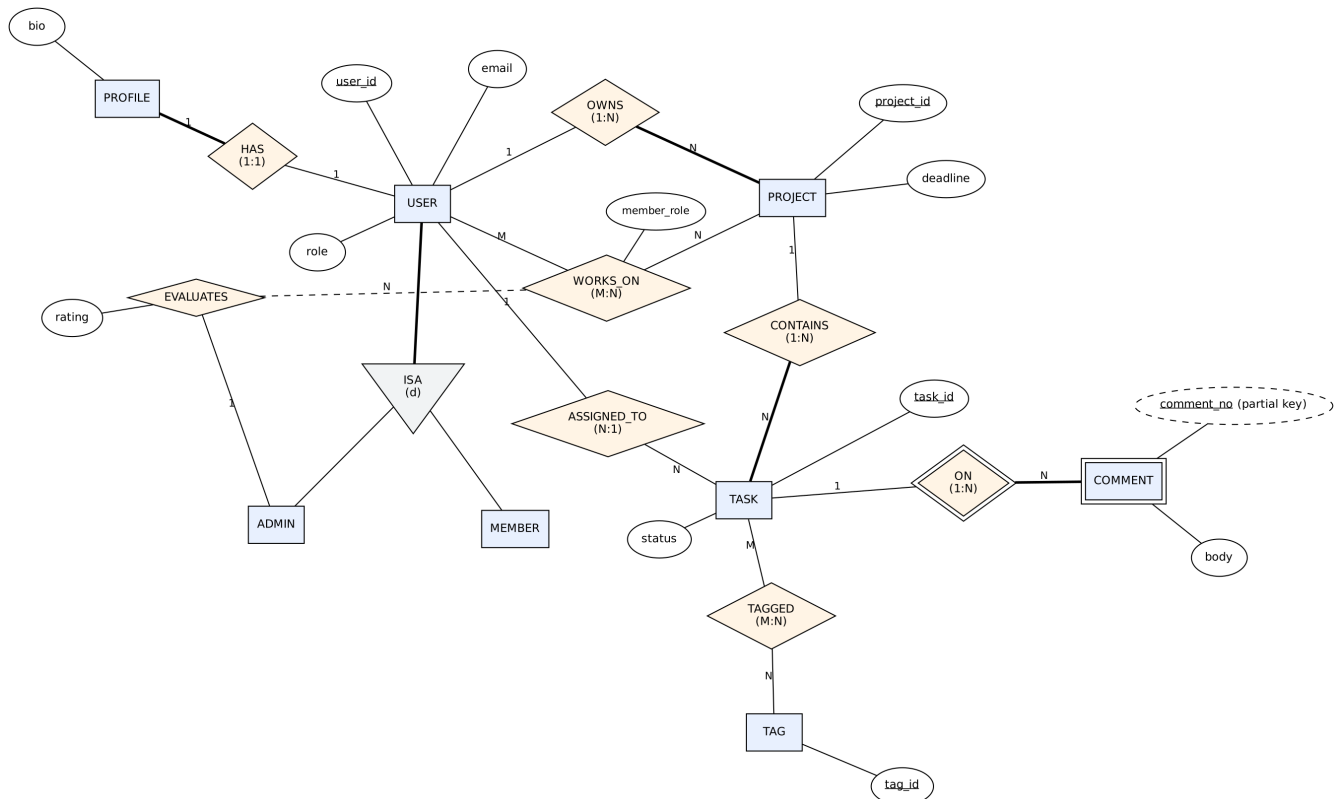
Aim

To model the Task Hub project with entities, attributes and relationships of every cardinality (1:1, 1:N, M:N), participation constraints, a weak entity, and the extended ER features specialisation/generalisation and aggregation, and then map the ER diagram to relational tables.

Program

Notation: rectangle = entity, double rectangle = weak entity, diamond = relationship, double diamond = identifying relationship, ellipse = attribute (underlined = key, dashed = partial key), thick line = total participation, ISA (d) = disjoint specialisation, dashed line = relationship on an aggregated relationship.

Task Hub ER diagram (generated from taskhub_er.dot with Graphviz)



s2.sql (ER-to-relational mapping)

```
-- Skilling 2: ER-to-Relational mapping of the Task Hub ER diagram
DROP DATABASE IF EXISTS taskhub_db;
CREATE DATABASE taskhub_db;
\c taskhub_db

-- Rule 1 (strong entity -> table): USER
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    role VARCHAR(10) NOT NULL CHECK (role IN ('ADMIN', 'MEMBER'))
);

-- Specialisation USER ISA {ADMIN, MEMBER} (disjoint, total):
-- one table per subclass, sharing the superclass primary key
```

```

CREATE TABLE admins (
    user_id      INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    access_level INT NOT NULL DEFAULT 1 CHECK (access_level BETWEEN 1 AND 3)
);
CREATE TABLE members (
    user_id      INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    skill_level  VARCHAR(12) NOT NULL DEFAULT 'BEGINNER'
);

-- Rule 3 (1:1 relationship HAS): foreign key with UNIQUE on the total side
CREATE TABLE profiles (
    profile_id SERIAL PRIMARY KEY,
    user_id    INT NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    bio        TEXT,
    github_url VARCHAR(200)
);

-- Rule 4 (1:N relationship OWNS): foreign key on the N side;
-- total participation of PROJECT -> NOT NULL
CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    owner_id   INT NOT NULL REFERENCES users(user_id),
    deadline   DATE
);

-- Rule 5 (M:N relationship WORKS_ON): separate table, composite primary key,
-- relationship attribute member_role stored in the relationship table
CREATE TABLE project_members (
    user_id      INT REFERENCES users(user_id) ON DELETE CASCADE,
    project_id   INT REFERENCES projects(project_id) ON DELETE CASCADE,
    member_role  VARCHAR(30) NOT NULL,
    joined_on    DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (user_id, project_id)
);

-- Aggregation: ADMIN EVALUATES the WORKS_ON relationship (user, project pair)
CREATE TABLE evaluations (
    evaluation_id SERIAL PRIMARY KEY,
    admin_id      INT NOT NULL REFERENCES admins(user_id),
    user_id       INT NOT NULL,
    project_id    INT NOT NULL,
    rating        INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    FOREIGN KEY (user_id, project_id) REFERENCES project_members(user_id, project_id)
);

-- 1:N CONTAINS (total participation of TASK) and N:1 ASSIGNED_TO (partial)
CREATE TABLE tasks (
    task_id      SERIAL PRIMARY KEY,
    project_id   INT NOT NULL REFERENCES projects(project_id) ON DELETE CASCADE,
    assignee_id  INT REFERENCES users(user_id) ON DELETE SET NULL,
    title        VARCHAR(200) NOT NULL,
    status       VARCHAR(12) NOT NULL DEFAULT 'TODO'
    CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    due_date     DATE
);

-- Rule 2 (weak entity COMMENT): primary key = owner key + partial key,
-- deleted together with its owner TASK
CREATE TABLE comments (
    task_id      INT REFERENCES tasks(task_id) ON DELETE CASCADE,
    comment_no   INT,
    author_id    INT NOT NULL REFERENCES users(user_id),
    body         TEXT NOT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (task_id, comment_no)
);

-- M:N TAGGED between TASK and TAG
CREATE TABLE tags (
    tag_id SERIAL PRIMARY KEY,
    name   VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE task_tags (
    task_id INT REFERENCES tasks(task_id) ON DELETE CASCADE,
    tag_id  INT REFERENCES tags(tag_id) ON DELETE CASCADE,
    PRIMARY KEY (task_id, tag_id)
);

```

```
-- Sample data
INSERT INTO users (full_name, email, role) VALUES
('Habeeb', 'habeeb@taskhub.dev', 'ADMIN'),
('Ananya Rao', 'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev', 'MEMBER');
INSERT INTO admins (user_id, access_level) VALUES (1, 3);
INSERT INTO members (user_id, skill_level) VALUES (2, 'INTERMEDIATE'), (3, 'BEGINNER');
INSERT INTO profiles (user_id, bio) VALUES (1, 'Backend lead'), (2, 'Frontend developer');
INSERT INTO projects (name, owner_id, deadline) VALUES ('Task Hub Backend', 1, '2026-11-30');
INSERT INTO project_members VALUES (2, 1, 'Database Engineer', '2026-09-01'),
(3, 1, 'API Developer', '2026-09-05');
INSERT INTO evaluations (admin_id, user_id, project_id, rating) VALUES (1, 2, 1, 5);
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES
(1, 2, 'Create PostgreSQL schema', 'IN_PROGRESS'),
(1, 3, 'Write FastAPI CRUD', 'TODO');
INSERT INTO comments (task_id, comment_no, author_id, body) VALUES
(1, 1, 1, 'Please add indexes on foreign keys'),
(1, 2, 2, 'Done, pushed to the repository');
INSERT INTO tags (name) VALUES ('database'), ('backend');
INSERT INTO task_tags VALUES (1, 1), (1, 2), (2, 2);

-- Verification 1: every relationship expressed as a foreign key
SELECT conrelid::regclass AS child_table,
       confrelid::regclass AS parent_table,
       pg_get_constraintdef(oid) AS definition
FROM   pg_constraint
WHERE  contype = 'f' AND connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::TEXT, confrelid::regclass::TEXT;

-- Verification 2: weak entity has a composite primary key
\d comments

-- Verification 3: 1:1 is enforced (a second profile for user 1 is rejected)
INSERT INTO profiles (user_id, bio) VALUES (1, 'Second profile');

-- Verification 4: aggregation - an evaluation must refer to an existing WORKS_ON pair
INSERT INTO evaluations (admin_id, user_id, project_id, rating) VALUES (1, 1, 1, 4);

-- Verification 5: deleting the owner removes the weak entities
DELETE FROM tasks WHERE task_id = 1;
SELECT COUNT(*) AS comments_left_for_task_1 FROM comments WHERE task_id = 1;

-- Verification 6: query across the M:N relationship
SELECT u.full_name, p.name AS project, pm.member_role, e.rating
FROM   project_members pm
JOIN   users u ON u.user_id = pm.user_id
JOIN   projects p ON p.project_id = pm.project_id
LEFT JOIN evaluations e ON e.user_id = pm.user_id AND e.project_id = pm.project_id
ORDER BY u.full_name;
```

Output

```
DROP DATABASE IF EXISTS taskhub_db;
DROP DATABASE
CREATE DATABASE taskhub_db;
CREATE DATABASE
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  role VARCHAR(10) NOT NULL CHECK (role IN ('ADMIN', 'MEMBER'))
);
CREATE TABLE
CREATE TABLE admins (
  user_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
  access_level INT NOT NULL DEFAULT 1 CHECK (access_level BETWEEN 1 AND 3)
);
CREATE TABLE
CREATE TABLE members (
  user_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
  skill_level VARCHAR(12) NOT NULL DEFAULT 'BEGINNER'
);
CREATE TABLE
CREATE TABLE profiles (
  profile_id SERIAL PRIMARY KEY,
  user_id INT NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
  bio TEXT,
```

```

        github_url VARCHAR(200)
    );
CREATE TABLE
CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    owner_id    INT NOT NULL REFERENCES users(user_id),
    deadline    DATE
);
CREATE TABLE
CREATE TABLE project_members (
    user_id     INT REFERENCES users(user_id) ON DELETE CASCADE,
    project_id  INT REFERENCES projects(project_id) ON DELETE CASCADE,
    member_role VARCHAR(30) NOT NULL,
    joined_on   DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (user_id, project_id)
);
CREATE TABLE
CREATE TABLE evaluations (
    evaluation_id SERIAL PRIMARY KEY,
    admin_id      INT NOT NULL REFERENCES admins(user_id),
    user_id       INT NOT NULL,
    project_id    INT NOT NULL,
    rating        INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    FOREIGN KEY (user_id, project_id) REFERENCES project_members(user_id, project_id)
);
CREATE TABLE
CREATE TABLE tasks (
    task_id      SERIAL PRIMARY KEY,
    project_id   INT NOT NULL REFERENCES projects(project_id) ON DELETE CASCADE,
    assignee_id  INT REFERENCES users(user_id) ON DELETE SET NULL,
    title        VARCHAR(200) NOT NULL,
    status       VARCHAR(12) NOT NULL DEFAULT 'TODO'
                CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    due_date     DATE
);
CREATE TABLE
CREATE TABLE comments (
    task_id      INT REFERENCES tasks(task_id) ON DELETE CASCADE,
    comment_no   INT,
    author_id    INT NOT NULL REFERENCES users(user_id),
    body         TEXT NOT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (task_id, comment_no)
);
CREATE TABLE
CREATE TABLE tags (
    tag_id SERIAL PRIMARY KEY,
    name   VARCHAR(30) NOT NULL UNIQUE
);
CREATE TABLE
CREATE TABLE task_tags (
    task_id INT REFERENCES tasks(task_id) ON DELETE CASCADE,
    tag_id  INT REFERENCES tags(tag_id) ON DELETE CASCADE,
    PRIMARY KEY (task_id, tag_id)
);
CREATE TABLE
INSERT INTO users (full_name, email, role) VALUES
('Habeeb', 'habeeb@taskhub.dev', 'ADMIN'),
('Ananya Rao', 'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev', 'MEMBER');
INSERT 0 3
INSERT INTO admins (user_id, access_level) VALUES (1, 3);
INSERT 0 1
INSERT INTO members (user_id, skill_level) VALUES (2, 'INTERMEDIATE'), (3, 'BEGINNER');
INSERT 0 2
INSERT INTO profiles (user_id, bio) VALUES (1, 'Backend lead'), (2, 'Frontend developer');
INSERT 0 2
INSERT INTO projects (name, owner_id, deadline) VALUES ('Task Hub Backend', 1, '2026-11-30');
INSERT 0 1
INSERT INTO project_members VALUES (2, 1, 'Database Engineer', '2026-09-01'),
                                    (3, 1, 'API Developer', '2026-09-05');
INSERT 0 2
INSERT INTO evaluations (admin_id, user_id, project_id, rating) VALUES (1, 2, 1, 5);
INSERT 0 1
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES
(1, 2, 'Create PostgreSQL schema', 'IN_PROGRESS'),
(1, 3, 'Write FastAPI CRUD', 'TODO');

```

```

INSERT 0 2
INSERT INTO comments (task_id, comment_no, author_id, body) VALUES
(1, 1, 1, 'Please add indexes on foreign keys'),
(1, 2, 2, 'Done, pushed to the repository');
INSERT 0 2
INSERT INTO tags (name) VALUES ('database'), ('backend');
INSERT 0 2
INSERT INTO task_tags VALUES (1, 1), (1, 2), (2, 2);
INSERT 0 3
SELECT conrelid::regclass AS child_table,
       confrelid::regclass AS parent_table,
       pg_get_constraintdef(oid) AS definition
FROM   pg_constraint
WHERE  contype = 'f' AND connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::TEXT, confrelid::regclass::TEXT;

```

child_table	parent_table	definition
admins	users	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
comments	tasks	FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE
comments	users	FOREIGN KEY (author_id) REFERENCES users(user_id)
evaluations	admins	FOREIGN KEY (admin_id) REFERENCES admins(user_id)
evaluations	project_members	FOREIGN KEY (user_id, project_id) REFERENCES project_members(user_id, project_id)
members	users	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
profiles	users	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
project_members	projects	FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE
project_members	users	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
projects	users	FOREIGN KEY (owner_id) REFERENCES users(user_id)
task_tags	tags	FOREIGN KEY (tag_id) REFERENCES tags(tag_id) ON DELETE CASCADE
task_tags	tasks	FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE
tasks	projects	FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE
tasks	users	FOREIGN KEY (assignee_id) REFERENCES users(user_id) ON DELETE SET NULL

(14 rows)

```

Table "public.comments"
  Column      |          Type          | Collation | Nullable |          Default
-----+-----+-----+-----+-----
 task_id      | integer                |           | not null |
 comment_no   | integer                |           | not null |
 author_id    | integer                |           | not null |
 body         | text                   |           | not null |
 created_at   | timestamp without time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "comments_pkey" PRIMARY KEY, btree (task_id, comment_no)
Foreign-key constraints:
    "comments_author_id_fkey" FOREIGN KEY (author_id) REFERENCES users(user_id)
    "comments_task_id_fkey" FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE

```

```

INSERT INTO profiles (user_id, bio) VALUES (1, 'Second profile');
ERROR:  duplicate key value violates unique constraint "profiles_user_id_key"
DETAIL:  Key (user_id)=(1) already exists.
INSERT INTO evaluations (admin_id, user_id, project_id, rating) VALUES (1, 1, 1, 4);
ERROR:  insert or update on table "evaluations" violates foreign key constraint "evaluations_user_id_project_id_fkey"
DETAIL:  Key (user_id, project_id)=(1, 1) is not present in table "project_members".
DELETE FROM tasks WHERE task_id = 1;
DELETE 1
SELECT COUNT(*) AS comments_left_for_task_1 FROM comments WHERE task_id = 1;

```

comments_left_for_task_1
0

(1 row)

```

SELECT u.full_name, p.name AS project, pm.member_role, e.rating
FROM   project_members pm
JOIN   users u ON u.user_id = pm.user_id
JOIN   projects p ON p.project_id = pm.project_id
LEFT JOIN evaluations e ON e.user_id = pm.user_id AND e.project_id = pm.project_id
ORDER BY u.full_name;

```

full_name	project	member_role	rating
Ananya Rao	Task Hub Backend	Database Engineer	5
Rahul Verma	Task Hub Backend	API Developer	

(2 rows)

Result

The ER diagram was mapped to 11 tables using the standard rules: strong entities became tables, the 1:1 HAS relationship became a UNIQUE foreign key, 1:N relationships became foreign keys on the N side (NOT NULL for total participation), M:N relationships became junction tables with composite keys, the weak entity COMMENT received the composite key (task_id, comment_no) with ON DELETE CASCADE, the ISA hierarchy became one table per subclass, and the aggregation was enforced by a composite foreign key to project_members. The verification statements confirmed that every rule is enforced by the database.